AI ASSISTED CODING ASSIGNMENT – 8.1

E. Shivani

2303A51098

BATCH-02

Task 1 (Password Strength Validator – Apply AI in Security Context)

Task: Apply AI to generate at least 3 assert test cases for is_strong_password(password) and implement the validator function

Requirements:

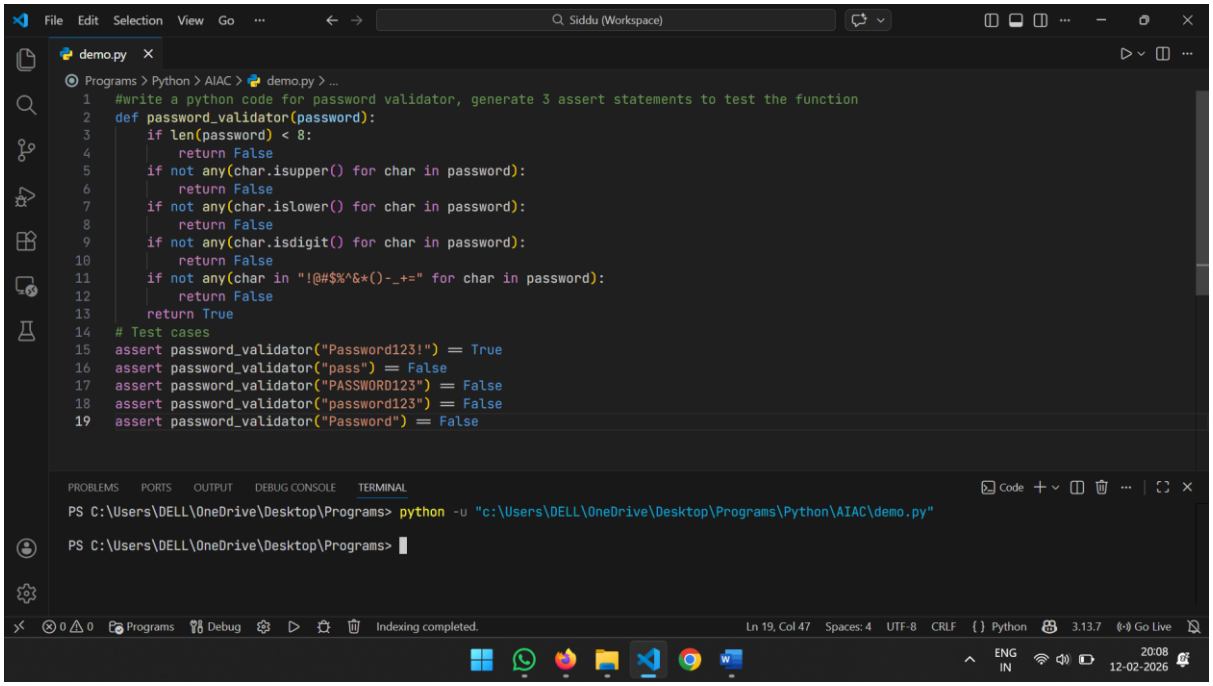
- Password must have at least 8 characters.
- Must include uppercase, lowercase, digit, and special character.
- Must not contain spaces.

Example Assert Test Cases:

- assert is_strong_password("Abcd@123") == True
- assert is_strong_password("abcd123") == False
- assert is_strong_password("ABCD@1234") == True

Expected Output:

- Password validation logic passing all AI-generated test cases.



Task 2 (Number Classification with Loops – Apply AI for Edge Case Handling)

Task: Use AI to generate at least 3 assert test cases for a classify_number(n) function. Implement using loops.

Requirements:

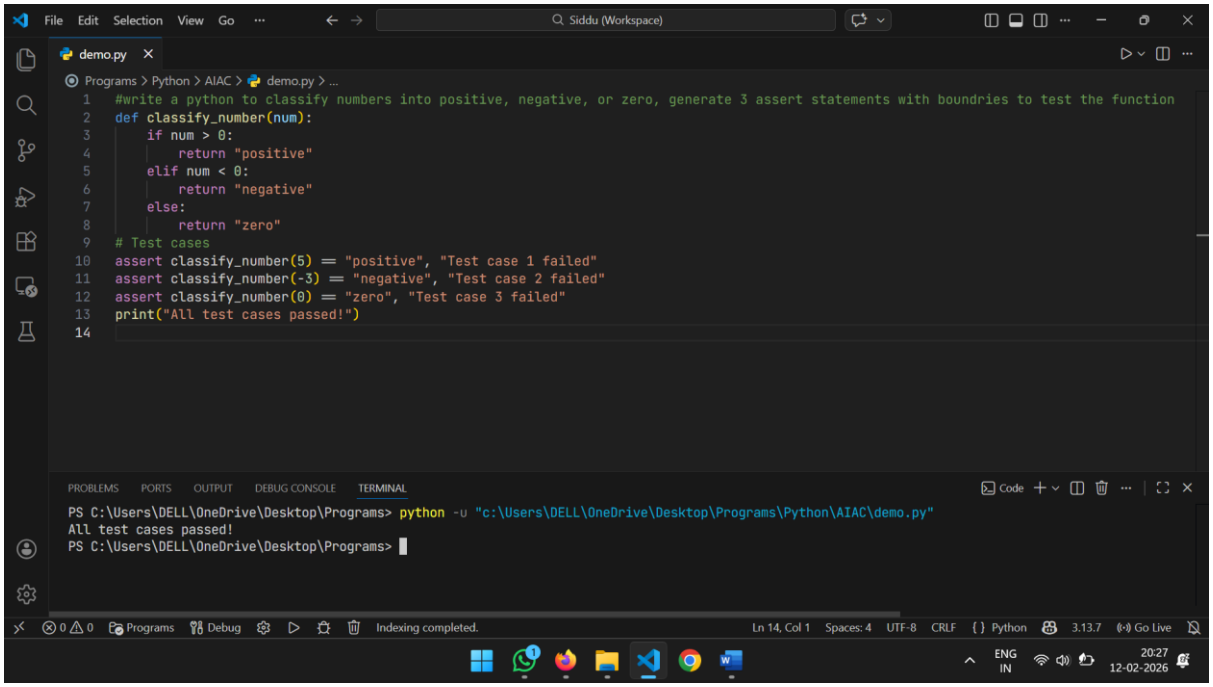
- Classify numbers as Positive, Negative, or Zero.
- Handle invalid inputs like strings and None.
- Include boundary conditions (-1, 0, 1).

Example Assert Test Cases:

- assert classify_number(10) == "Positive"
- assert classify_number(-5) == "Negative"
- assert classify_number(0) == "Zero"

Expected Output:

- Classification logic passing all assert tests.



Task 3 (Anagram Checker – Apply AI for String Analysis)

Task: Use AI to generate at least 3 assert test cases for is_anagram(str1, str2) and implement the function.

Requirements:

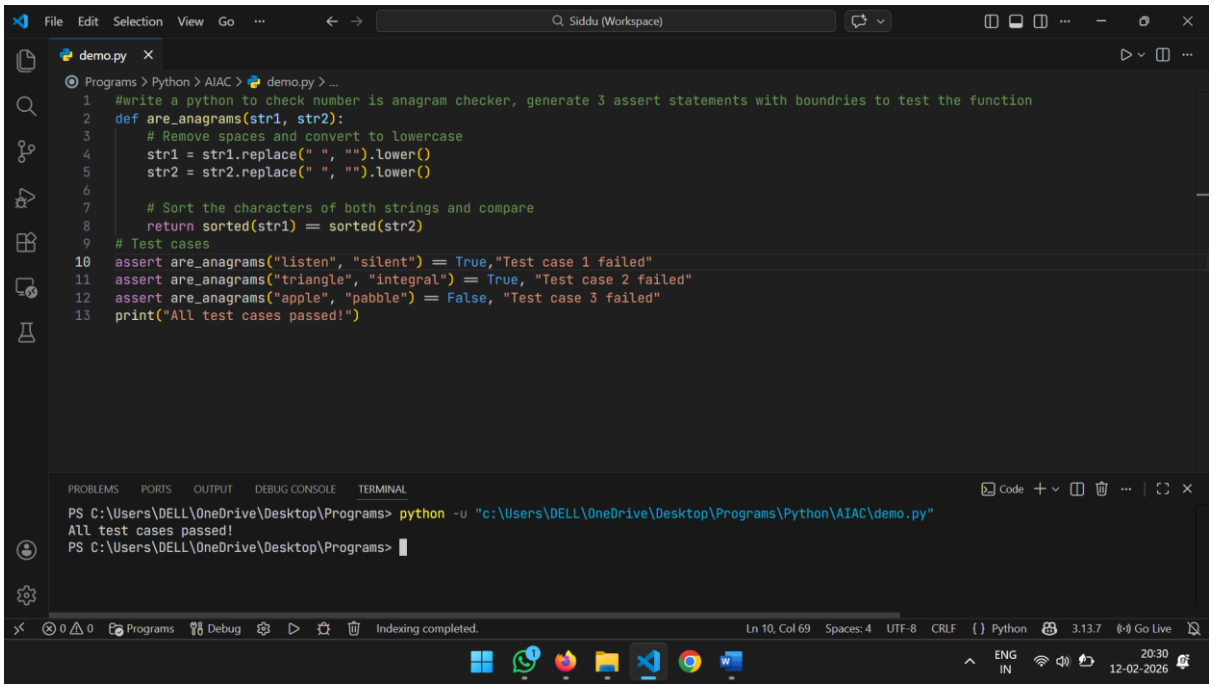
- Ignore case, spaces, and punctuation.
- Handle edge cases (empty strings, identical words).

Example Assert Test Cases:

- assert is_anagram("listen", "silent") == True
- assert is_anagram("hello", "world") == False
- assert is_anagram("Dormitory", "Dirty Room") == True

Expected Output:

- Function correctly identifying anagrams and passing all AI-generated tests.



Task 4 (Inventory Class – Apply AI to Simulate Real-World Inventory System)

Task: Ask AI to generate at least 3 assert-based tests for an Inventory class with stock management.

Requirements:

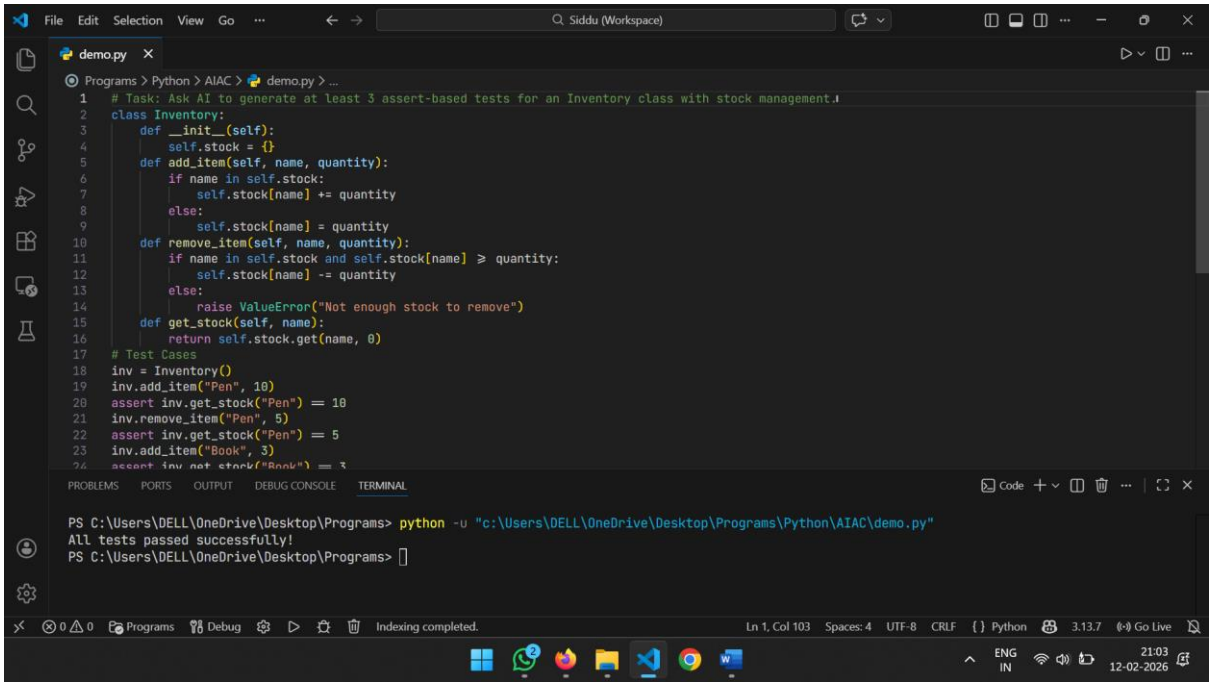
- add_item(name, quantity)
- remove_item(name, quantity)
- get_stock(name)

Example Assert Test Cases:

- nv = Inventory()
- inv.add_item("Pen", 10)
- assert inv.get_stock("Pen") == 10
- inv.remove_item("Pen", 5)
- assert inv.get_stock("Pen") == 5
- inv.add_item("Book", 3)
- assert inv.get_stock("Book") == 3

Expected Output:

- Fully functional class passing all assertions.



Task 5 (Date Validation & Formatting – Apply AI for Data Validation)

Task: Use AI to generate at least 3 assert test cases for validate_and_format_date(date_str) to check and convert dates.

Requirements:

- Validate "MM/DD/YYYY" format.
- Handle invalid dates.
- Convert valid dates to "YYYY-MM-DD"

Example Assert Test Cases:

- assert validate_and_format_date("10/15/2023") == "2023-10-15"
- assert validate_and_format_date("02/30/2023") == "Invalid Date"
- assert validate_and_format_date("01/01/2024") == "2024-01-01"

Expected Output:

- Function passes all AI-generated assertions and handles edge cases.

demo.py

Programs > Python > AIAC > demo.py > ...

```python
def validate_and_format_date(date_str):
    try:
        for fmt in ('%Y-%m-%d', '%d/%m/%Y', '%m-%d-%Y'):
            try:
                date_obj = datetime.strptime(date_str, fmt)
                return date_obj.strftime('%Y-%m-%d')  # Return in standard format
            except ValueError:
                continue
        raise ValueError("Date format not recognized")
    except ValueError as e:
        return str(e)

# Test cases
assert validate_and_format_date("2024-06-15") == "2024-06-15"  # Valid ISO format
assert validate_and_format_date("15/06/2024") == "2024-06-15"  # Valid European format
assert validate_and_format_date("06-15-2024") == "2024-06-15"  # Valid US format
assert validate_and_format_date("2024/06/15") == "Date format not recognized"  # Invalid format
assert validate_and_format_date("June 15, 2024") == "Date format not recognized"  # Invalid format
assert validate_and_format_date("2024-15-06") == "Date format not recognized"  # Invalid format
print("All test cases passed!")
```

PROBLEMS PORTS OUTPUT DEBUG CONSOLE TERMINAL

```
PS C:\Users\DELL\OneDrive\Desktop\Programs> python -u "c:\Users\DELL\OneDrive\Desktop\Programs\Python\AIAC\demo.py"
All test cases passed!
PS C:\Users\DELL\OneDrive\Desktop\Programs>
```

Indexing completed. Ln 3, Col 1 Spaces: 4 UTF-8 CRLF Python 3.13.7 Go Live

ENG IN 21:06 12-02-2026